Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2025-26

Class:	**TE**	**Semester:**	**VI**
Course Code:	**CSL603**	**Course Name:**	**SEPM**

Name of Student:	**Shravani S. Raut**
Roll No.:	**40**
Experiment No.:	**05**
Title of the Experiment:	**To create a Jenkins pipeline that automates the build, test, and deployment of a Maven- based Java application on an Apache Tomcat server.**
Date of Performance:	**11/02/2026**
Date of Submission:	**18/02/2026**

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Meet Expect Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty: Ms Rujuta Vartak
Signature:
Date:

Experiment No-05

Aim: To create a Jenkins pipeline that automates the build, test, and deployment of a Maven-based Java application on an Apache Tomcat server.

Objective:

- Set up **Jenkins** on Windows.
- Configure **Maven** and **Tomcat** for automated deployment.
- Create a **Jenkins pipeline script** to perform:
    1. **Checkout Code** from GitHub
    2. **Build the project** using Maven
    3. **Run Tests**
    4. **Deploy the application** to Tomcat
    5. **Restart Tomcat** for changes to take effect
- Automate the pipeline using **GitHub Webhooks**.

Theory:

1. Jenkins:

Jenkins is an open-source automation server that helps in continuous integration and continuous deployment (CI/CD). It allows developers to automate build, test, and deployment processes.

2. Maven:

Maven is a build automation tool primarily used for Java projects. It helps manage dependencies, compile source code, run tests, and package the project into a WAR file.

3. Apache Tomcat:

Tomcat is a Java servlet container that runs Java-based web applications. Deploying a WAR file to Tomcat makes the application accessible via a web browser.

4. Jenkins Pipeline:

A Jenkins Pipeline is a set of instructions that defines the CI/CD workflow. The pipeline stages typically include:

1. Checkout Code - Cloning the latest code from GitHub.
2. Build with Maven - Compiling and packaging the application.
3. Run Tests - Ensuring the application is stable.

4. Deploy to Tomcat - Copying the built WAR file to the Tomcat server.
5. Restart Tomcat - Applying the

changes. Steps for implementation:

Step 1: Follow the steps of experiment no -4 for setup environments like maven and jdk

Step : Install Apache Tomcat

- Download Tomcat 9 or 10 from Tomcat Download Page.
- Extract it to C:\tomcat.
- To start Tomcat,

run: Cmd:

C:\tomcat\bin\startup.bat

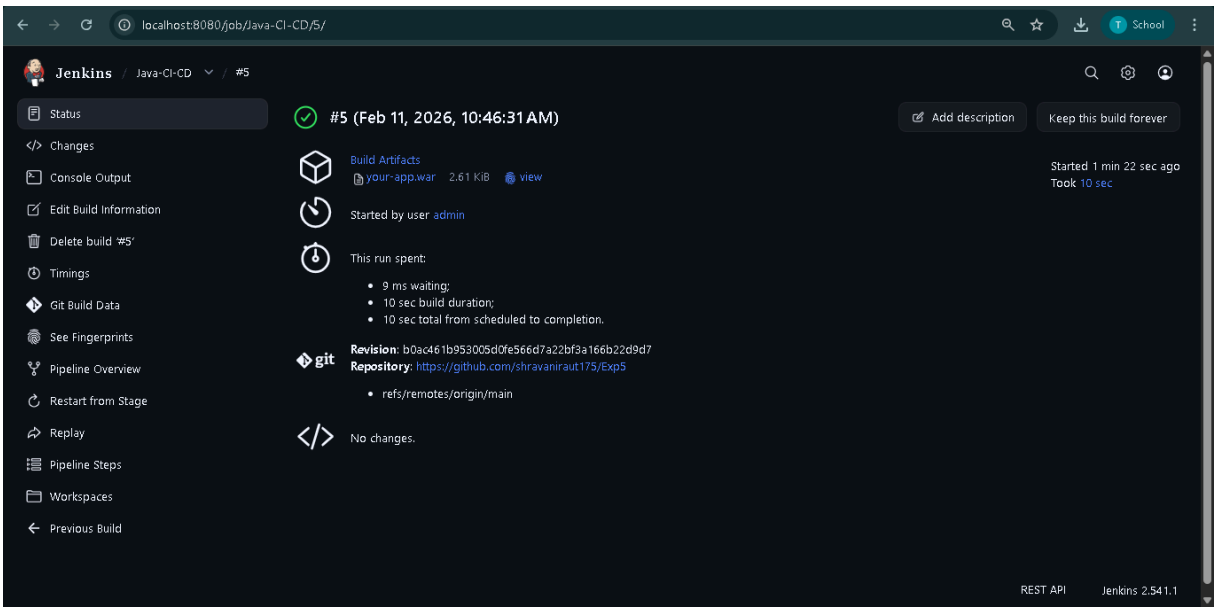
- Verify by

opening: Web browser:

http://localhost:8080

step 3: Create a New Jenkins Pipeline Job

1. Click "New Item".
2. Enter a Job Name and select "Pipeline".
3. Click OK.
4. Scroll down to the Pipeline section.
5. Select Pipeline Script.



Step 4: Add Pipeline Script

Paste the following script in the Pipeline section:

File type groovy

```
pipeline {
    agent any

    environment {
        MAVEN_HOME = 'C:\\maven' // Set the Maven path
        TOMCAT_WEBAPPS = 'C:\\tomcat\\webapps' // Path to Tomcat webapps directory
    }

    stages {
        stage('Checkout Code') {
            steps {
                git 'https://github.com/your-repo/your-app.git' // Replace with your repository
            }
        }
```

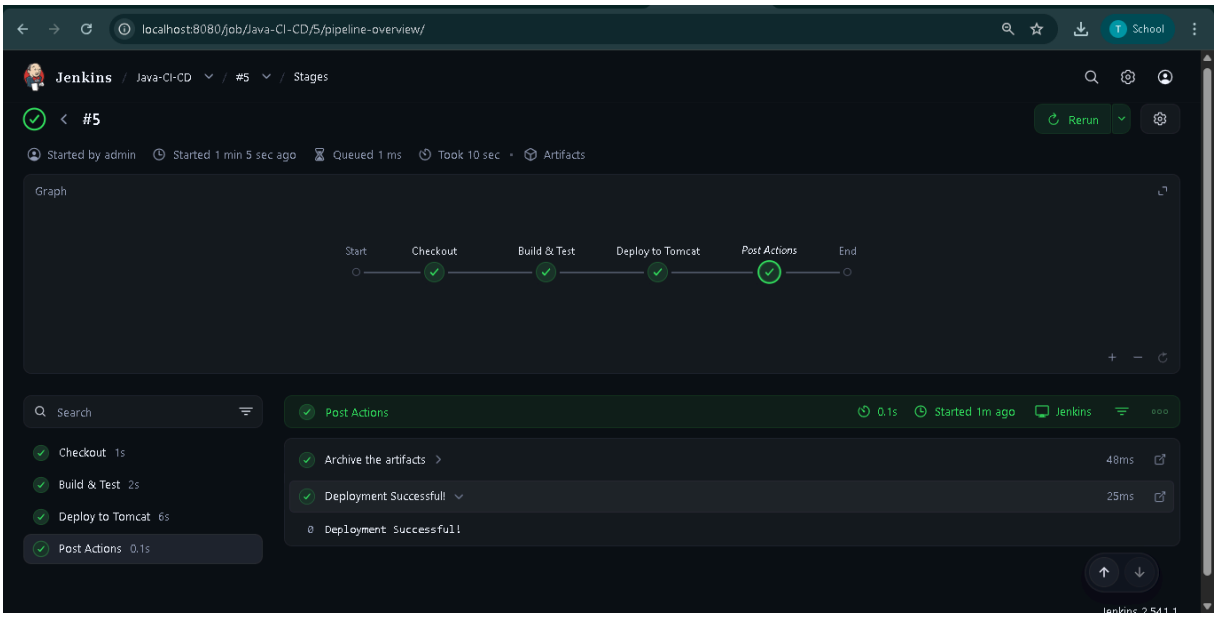
```
    stage('Build with Maven') {
        steps {
            bat "%MAVEN_HOME%\\bin\\mvn clean package"
        }
    }

    stage('Run Tests')
        { steps {
            bat "%MAVEN_HOME%\\bin\\mvn test"
        }
    }

    stage('Deploy to Tomcat') {
        steps {
            bat "copy target\\*.war
            %TOMCAT_WEBAPPS%\\your-app.war" bat
            "C:\\tomcat\\bin\\shutdown.bat"
            bat "C:\\tomcat\\bin\\startup.bat"
        }
    }
}

post {
    success {
        echo 'Deployment Successful!'
    }
    failure {
        echo 'Build or Deployment Failed!'
```

```
        }
    }
}
```



Step 5: Save and Run the Pipeline

1. Click "Save".
2. Click "Build Now".
3. Jenkins will pull the code, build, test, and deploy the application.

Step 6: Verify the Deployment

1. Open a browser and visit:

   http://localhost:8080/your-a

   p p
2. Your Java application should now be running! 🎉

Step 7: Automate the Pipeline with Webhook

If you want Jenkins to automatically build the project when code is updated in GitHub:

Configure GitHub Webhook:

1. Go to GitHub Repository → Settings → Webhooks.
2. Click "Add webhook".
3. Set the Payload URL as:

   http://your-jenkins-server:8080/github-webhook/
4. Select application/json as the content type.
5. Choose "Push event".
6. Click "Add Webhook".

Conclusion:

1. How does Tomcat differ from other web servers like Apache HTTP Server or Nginx?
   Tomcat is different from traditional web servers like Apache HTTP Server or Nginx because it is specifically designed for running **Java-based applications**. Here's how they differ:

   Tomcat

   Type: Tomcat is an application server (Servlet and JSP container).

   Purpose: It is used to run Java-based web applications such as Servlets and JSP.

   Best For: Java applications like Spring and Hibernate.

   Processing Model: It uses Java Virtual Machine (JVM) to execute applications.

   Performance: Optimized for Java application deployment.

   Use Case: Used for running Servlets, JSP, and Java web frameworks.

   Apache HTTP Server

   Type: Apache HTTP Server is a general-purpose web server.

   Purpose: It serves both static and dynamic content (like PHP, Python, CGI).

   Best For: Hosting websites and running server-side scripts.

   Processing Model: Process-based model (creates new processes or threads for requests).

   Performance: Efficient for static content and flexible configurations.

   Use Case: Used for hosting websites and handling PHP or Perl scripts.

Nginx

Type: Nginx is a web server and reverse proxy server.

Purpose: It handles HTTP requests, load balancing, and reverse proxying.

Best For: High-performance static content and handling heavy traffic.

Processing Model: Event-driven and non-blocking I/O model.

Performance: Very fast and efficient for concurrent connections.

Use Case: Used for load balancing, reverse proxy, and serving high-traffic websites.

2. What are the main components of Tomcat?

Apache Tomcat consists of several components that help in handling Java web applications. The key components are:

1. **Catalina (Servlet Container)**
   - Tomcat's core component that **implements the Servlet API**.
   - Manages **request processing, session handling, and lifecycle management** of Java servlets.
2. **Coyote (HTTP Connector)**
   - Acts as the **HTTP web server** for processing incoming client requests.
   - Supports **HTTP 1.1, HTTP/2, and AJP (Apache JServ Protocol)**.
3. **Jasper (JSP Engine)**
   - Tomcat's **JSP (JavaServer Pages) engine** that converts JSP files into servlets.
   - Supports JSP EL (Expression Language) and JSP Tag Libraries.
4. **Cluster**
   - Provides **load balancing and session replication** in distributed environments.
   - Helps in handling high availability and failover scenarios.
5. **Valves & Realm**
   - **Valves** – Used for **logging, security filtering, and access control**.
   - **Realm** – Manages **authentication and user role mapping** (supports JDBC, LDAP, and MemoryRealm).
6. **Web Applications (Host & Context)**
   - **Host** – Represents a virtual host inside Tomcat (e.g., multiple domains on a single Tomcat instance).
   - **Context** – Defines the specific web application's configuration (`web.xml`).
7. **Tomcat Manager**
   - A **web-based admin tool** for managing applications (deploying, starting, stopping)